Design a Chat System - Step 1: Understand the Problem and Establish Design Scope

1. Requirement Clarification

Before designing a chat system, it is extremely important to understand:

- Type of chat application
- Scale of the system
- Features required
- Performance expectations

Different chat applications focus on different requirements:

Chat Application	Main Focus
WhatsApp / Messenger	One-to-one chat
Slack	Office group communication
Discord	Large groups + voice chat
WeChat	Multi-purpose communication

2. Candidate–Interviewer Discussion

Type of Chat

Candidate: What kind of chat app shall we design? One-on-one or group based?

Interviewer: The system should support both one-on-one and group chat.

Platform Support

Candidate: Is this a mobile app, web app, or both?

Interviewer: Both.

System Scale

Candidate: What is the expected scale of the application?

Interviewer: The system should support **50 million daily active users (DAU).**

Group Size

Candidate: What is the maximum group size?

Interviewer: Maximum of **100 users per group**.

Features Required

Candidate: What features are important? Are attachments supported?

Interviewer: Important features are:

- One-on-one chat
- Group chat
- Online presence indicator

Only **text messages** are supported.

Message Size

Candidate: Is there a message size limit?

Interviewer: Yes. Maximum text length is:

```None
100,000 characters
```

Encryption

Candidate: Is end-to-end encryption required?

Interviewer: Not required for now.

Message History

Candidate: How long should chat history be stored?

Interviewer: Forever.

3. Final Requirements

After clarification, the system requirements become:

Functional Requirements

One-on-One Chat

- Real-time messaging
- Low latency delivery

Group Chat

- Maximum group size = 100 users

Online Presence

- Show whether a user is online/offline

Multiple Device Support

- Same account can log in from:
  - Mobile
  - Web
  - Desktop
- Messages should sync across devices

Push Notifications

- Notify users about new messages

Text Messaging Only

- No media files
- No image/video sharing

Permanent Chat History

- Messages stored forever

4. Non-Functional Requirements

Low Latency

Messages should be delivered almost instantly.

Scalability

System must support:

```None
50 million DAU
```

Reliability

- Messages should not be lost
- System should recover from failures

High Availability

Chat service should remain operational even during failures.

5. Important Design Challenges

Persistent Connections

Real-time chat requires maintaining long-lived connections between clients and servers.

Distributed System Complexity

Need to handle:

- Millions of concurrent users
- Multiple servers
- Synchronization across devices

Data Storage

Since messages are stored forever:

- Large-scale storage system required
- Efficient retrieval needed

Notification Coordination

Push notifications must work correctly when users are offline.

6. Design Scope for This Chapter

The design will focus on:

Feature	Included
One-on-one chat	✅

Group chat	✅
Online indicator	✅
Multi-device support	✅
Push notifications	✅
Text messages	✅
Media sharing	❌
End-to-end encryption	❌ (for now)

Key Takeaway

Requirement clarification is critical because chat system architecture changes significantly depending on:

- One-on-one vs group chat
- Small scale vs massive scale
- Media support
- Encryption requirements

For this design, the focus is on:

- Real-time messaging
- Low latency
- Scalability for 50 million DAU
- Reliable multi-device communication